SECURE FILE TRANSFER MONITORING SYSTEM

Project Documentation and Technical Report

1. Abstract

A local-first defensive application that monitors filesystem activity in user-selected folders, records file events, calculates SHA-256 hashes, flags potentially sensitive movements, stores metadata in SQLite and generates audit reports.

2. Motivation

Unauthorized copying, accidental data exposure, tampering and deletion create risks for organizations. Monitoring selected folders improves visibility and supports investigation.

3. Objectives

Record create, modify, move and delete events; detect sensitive extensions and filename keywords; verify file integrity using SHA-256; generate alerts; store audit logs; export reports.

4. Tools

Python, Streamlit, Watchdog, hashlib, SQLite, Pandas, Plotly and ReportLab.

5. Architecture

The user selects a folder. Watchdog observes filesystem events. The handler classifies events, hashes readable files, checks sensitivity and destination rules, then stores events and alerts in SQLite. Streamlit displays metrics and reports.

6. Modules

app.py: dashboard; src/monitor.py: watcher; src/integrity.py: SHA-256; src/policy.py: file policy; src/database.py: SQLite; src/reports.py: PDF report; tests/test_core.py: unit tests.

7. Security and Privacy

Only explicitly selected folders are monitored. The application does not execute monitored files or upload their contents. Metadata and hashes are stored locally. Use only on systems you are authorized to monitor.

8. Testing

Automated tests cover hash changes, sensitive file detection and destination allowlist matching. Manually test file creation, modification, movement and deletion using harmless test files.

9. Limitations

Filesystem events can be missed or coalesced. The prototype does not comprehensively detect USB, network, browser or cloud-sync transfers and does not reliably identify users/processes. Streamlit Cloud cannot monitor a personal laptop. UI policy settings are session-scoped.

10. Future Enhancements

Persistent policy configuration, platform-specific USB monitoring, process attribution, network integrations, authentication, notifications and dedicated background service.

11. Conclusion

The project demonstrates defensive filesystem monitoring, integrity verification, alerting and audit reporting. It is an educational prototype and requires additional platform-specific validation before production use.